

ROS2 Day3-hw1 과제 보고서

로봇 21기 예비단원 최윤서

1. 과제 조건 이해

- ROS2 1일차 과제 3번 패키지에서 굵기랑 색상 파라미터 설정
→ yaml파일에서 불러오는 형식으로 변환
- 작동 영상에 yaml 파일에서 파라미터 변경하는 전후 과정을 한 영상으로 포함해야됨

2. 코드 구조

```
ROS2_Day3_hw1_pkg
├── CMakeLists.txt
├── package.xml
├── config
│   └── params.yaml
├── include
│   └── ROS2_Day3_hw1_pkg
│       └── turtle_shape.hpp
├── launch
│   └── robot.launch.py
└── src
    └── turtle_shape.cpp
```

3. 코드 구현

3-1. 코드 동작 과정

- 1) 키보드 입력(WASD) → 도형 선택
- 2) params.yaml에서 설정한 색상, 굵기 적용
- 3) SetPen 서비스 호출
- 4) Twist 메시지 발행
- 5) turtlesim에서 도형 그리기

3-2. ROS2 노드 및 publisher 설정

- ROS2 노드 생성

```
class TurtleShape : public rclcpp::Node
{
public:
    TurtleShape();
    ~TurtleShape();
```

- Publisher 생성

```
publisher_ = this->create_publisher<geometry_msgs::msg::Twist>(
    "/turtle1/cmd_vel",
    10
);
```

**subscriber는 왜 없는가

- /turtle1/cmd_vel 에서 publish하고 turtlesim이 그 명령을 받아서 움직임
→ subscriber가 필요 없음

3-3. yaml 파라미터 설정

- 기존 코드에 직접 저장되어있던 도형별 색상, 굵기를 yaml파일의 파라미터로 구분
- 소스 코드 수정 X → yaml파일의 값만 변경 → 도형의 색상과 팬 굵기를 변경할 수 있도록함.

```
turtle_shape:
  ros__parameters:

    square_r: 0
    square_g: 255
    square_b: 255
    square_width: 10

    triangle_r: 255
    triangle_g: 255
    triangle_b: 0
    triangle_width: 20

    circle_r: 255
    circle_g: 0
    circle_b: 255
    circle_width: 15
```

- 노드 이름: traffic

3-4. yaml 에서 파라미터를 설정

- C++에서 파라미터 선언

```
this->declare_parameter("square_r", 0);  
this->declare_parameter("square_g", 255);  
this->declare_parameter("square_b", 0);  
this->declare_parameter("square_width", 5);
```

- 선언한 파라미터 값 가져오기

```
this->get_parameter("square_r", square_r_);  
this->get_parameter("square_g", square_g_);  
this->get_parameter("square_b", square_b_);  
this->get_parameter("square_width", square_width_);
```

- declare_parameter → ROS2 파라미터 선언
- get_parameter → yaml 파일에서 설정된 값을 프로그램 내부 변수로 가져옴

3-5. 키보드 입력에 따른 도형 선택

```
else if (key == "W")  
{  
    shape_ = "square";  
  
    pen_r_ = square_r_;  
    pen_g_ = square_g_;  
    pen_b_ = square_b_;  
    pen_width_ = square_width_;  
  
    std::cout << "사각형 선택" << std::endl;  
}
```

- W: 사각형
- S: 삼각형
- A: 원
- D: 종료

3-6. SetPen → 색상 및 굵기 적용

```
request->r = pen_r_;  
request->g = pen_g_;  
request->b = pen_b_;  
request->width = pen_width_;  
request->off = 0;
```

- 선택된 도형에 따라 해당하는 RGB 값 호출
- 마찬가지로 선택된 도형에 따라 펜 굵기 서비스 요청 호출

3-7. Timer를 활용한 도형 제어

- 10ms 주기의 Timer를 사용하여 현재 도형의 상태 확인
- /turtle/cmd_vel로 속도 명령을 지속적으로 발행
- 각 상태에서 경과 시간 확인 → 직진과 회전 동작을 순차적으로 수행

****왜 각도 계산을 안하고 Timer를 사용했는가?**

수습 단원 때 진행한 라인트레이서 프로젝트에서 평행사변형 구간을 타이머로 일정 시간만큼 회전하고 직진하는 코드를 구현하고자했습니다.
하지만, 시도할 때마다 그리고 날마다 모터의 속도가 미세하게 달라져서 일정한 시간만큼 회전해도 회전하는 각도가 달라졌습니다. 따라서 이번 과제에서 소프트웨어적으로는 정확하게 동작할 수 있는지 테스트해보고싶어 Timer를 활용해서 코드 구현해봤습니다.

4. 실행 방법

4-1. 패키지 빌드

```
cd ~/ros2_ws  
colcon build --packages-select ROS2_Day3_hw1_pkg
```

4-2. 빌드 후

```
source install/setup.bash
```

4-3. 터미널 1 turtlesim 실행

```
source /opt/ros/$ROS_DISTRO/setup.bash  
ros2 run turtlesim turtlesim_node
```

4-4. 터미널 2 과제 노드 실행

```
cd ~/ros2_ws  
source install/setup.bash  
  
ros2 run ROS2_Day3_hw1_pkg turtle_shape --ros-args \  
--params-file ~/ros2_ws/src/ROS2_Day3_hw1_pkg/config/params.yaml
```